

SupportLens

Architecture & Technical Documentation

Version 1.0.0 · SupportLens Engineering Team · June 2026

SupportLens is a full-stack, AI-powered customer support platform built on WebRTC SFU architecture. It delivers real-time video/audio, persistent chat, AI-generated session summaries, and a live admin dashboard — designed for hackathon demonstration yet architecturally sound for production evolution.

Next.js 16	React 19	Mediasoup SFU	Socket.IO 4	Prisma + SQLite	Google Gemini
------------	----------	------------------	-------------	--------------------	------------------

Table of Contents

1.	Project Overview	3
2.	System Architecture	3
3.	Component Architecture	4
4.	Request Flow	5
5.	Database Design	6
6.	Deployment Architecture	7
7.	Demo Credentials	8
8.	Key Features	8
9.	Technical Highlights	9
10.	Future Enhancements	11
A.	Technology Reference	12

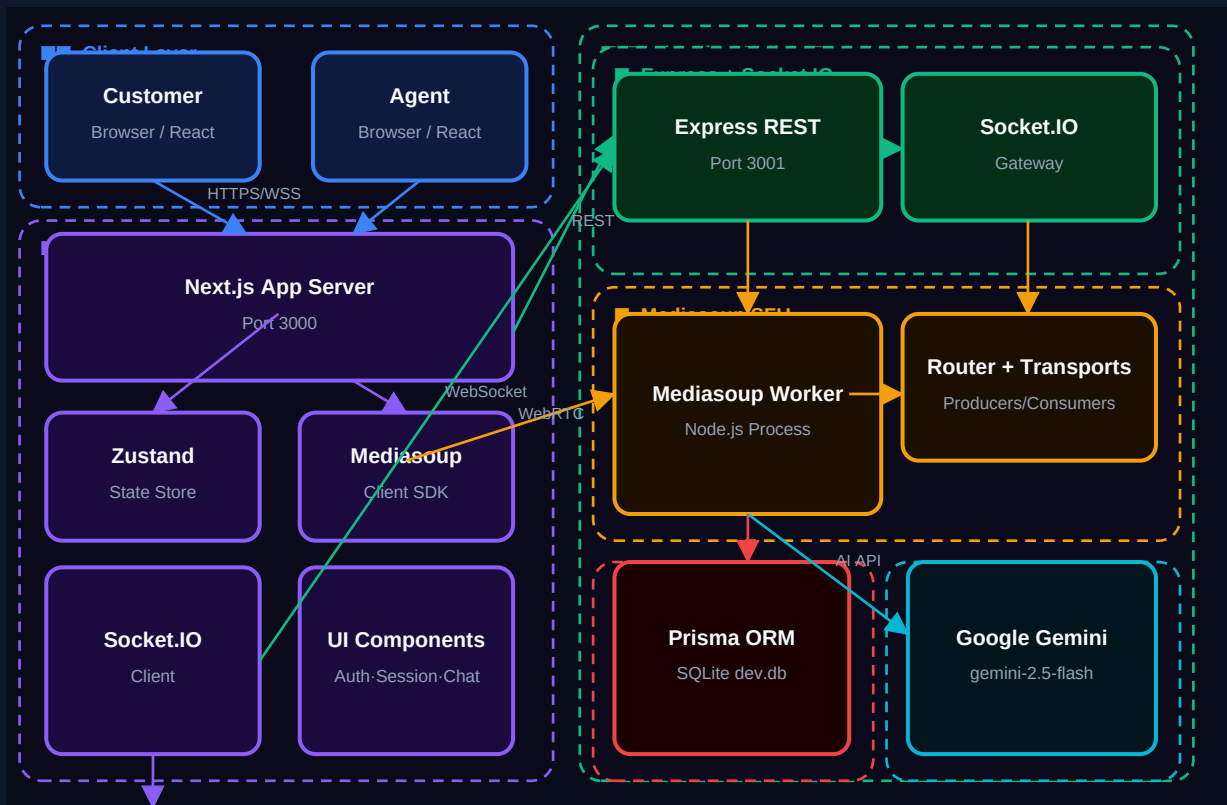
1. Project Overview

SupportLens is a full-stack, AI-powered customer support platform engineered for real-time, high-fidelity communication between support agents and their customers. Built on a modern WebRTC SFU (Selective Forwarding Unit) architecture, SupportLens eliminates the latency and complexity of peer-to-peer media while delivering a production-grade feature set in a single cohesive platform.

■ Real-Time Media	WebRTC video + audio routed through a Mediasoup SFU — eliminating mesh topology limitations.
■ Persistent Chat & File Sharing	Powered by Socket.IO with all messages and file metadata durably stored in SQLite via Prisma ORM.
■ AI-Generated Session Summaries	Google Gemini API automatically analyzes transcripts and produces structured support call reports.
■ Operational Observability	Live admin dashboard surfacing active sessions, participant counts, message volumes, and server uptime.
■ Resilient Reconnect & Recovery	Gracefully handles customer disconnections without disrupting the agent session.

2. System Architecture

The diagram below illustrates the high-level system topology — how client browsers, the Next.js application layer, the Express signaling backend, the Mediasoup SFU, the persistence layer, and external AI services interconnect.



End-to-end system architecture. Media flows via WebRTC SRTP directly between client browsers and the Mediasoup SFU worker; signaling, chat, and file events traverse the Socket.IO gateway; and AI summary requests are dispatched from the Express REST layer to the Google Gemini API.

3. Component Architecture

This section decomposes SupportLens into constituent components across the frontend, backend, and data layers, along with external service integrations.

Frontend Layer

Authentication UI	Lobby + Role Selector with Demo Credential Verification
Session UI	CallWindow.tsx · VideoStream Component
Chat UI	ChatPanel.tsx · File Upload + Preview
Control Panel	Mute / Camera / Hang Up controls
useMediasoupClient Hook	WebRTC Orchestration — transport, produce, consume lifecycle
useSocket Hook	Singleton Socket.IO connection management
Zustand Store	Streams · Peers · Chat · Mute/Video State · Identity
Admin Dashboard	Real-Time Stats · Session List · AI Summary Panel

Backend Layer

GET /health	Health check endpoint
POST /api/upload	Multer file validation (PNG/PDF ≤10MB)
GET /api/sessions	Paginated session list
POST /api/sessions/:id/summary	Gemini AI summarization trigger
GET /api/admin/stats	Live operational metrics
Socket.IO Gateway	join · produce · consume · sendMessage · endCall events
Room Manager	In-Memory Peer & Transport State
Mediasoup Service	Worker · Router · Transport Factory

Data Layer

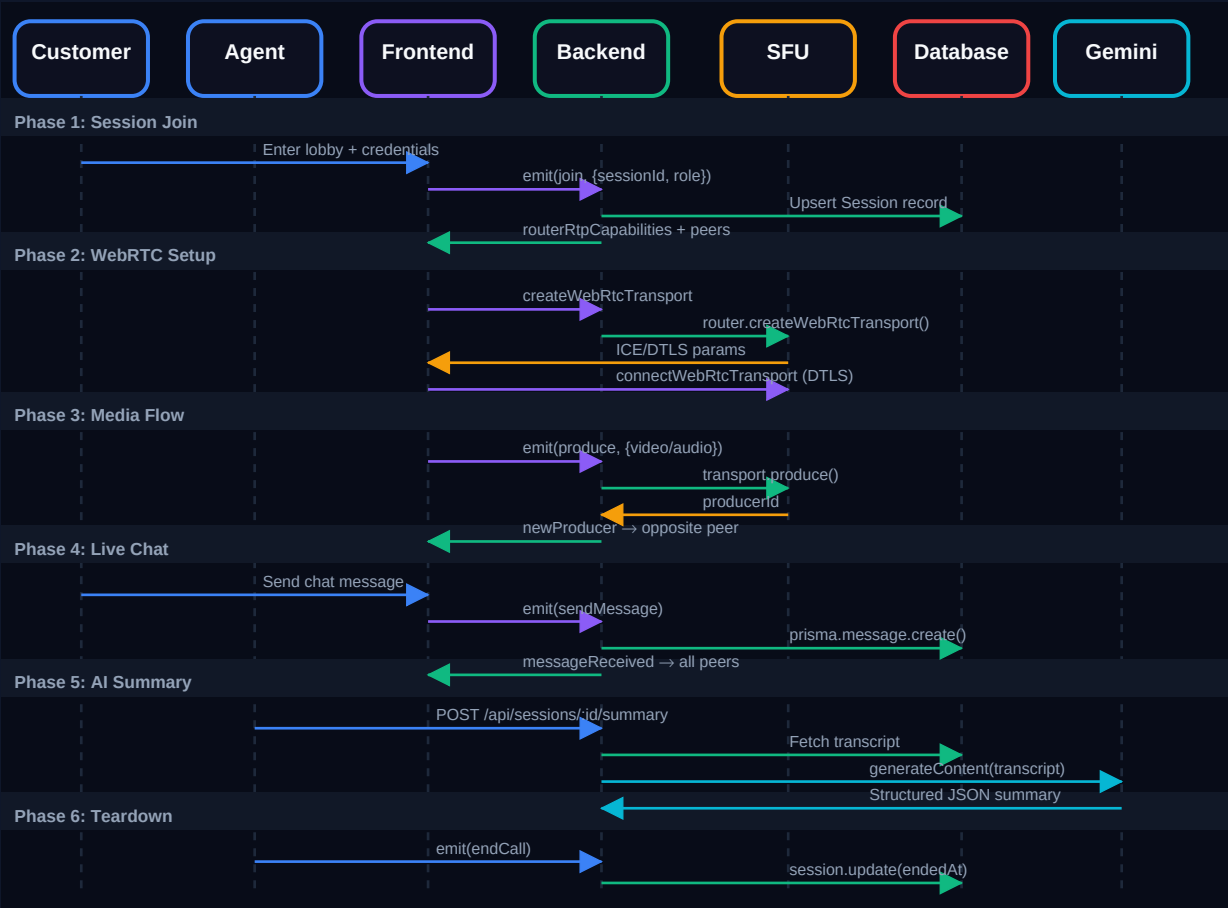
Prisma ORM	Type-safe query layer with schema migrations
SQLite Database	prisma/dev.db — embedded relational persistence
Static File Store	/public/uploads/ — disk-stored images and PDFs

External Services

Google Gemini API	gemini-2.5-flash · Structured JSON response with responseSchema
-------------------	---

4. Request Flow

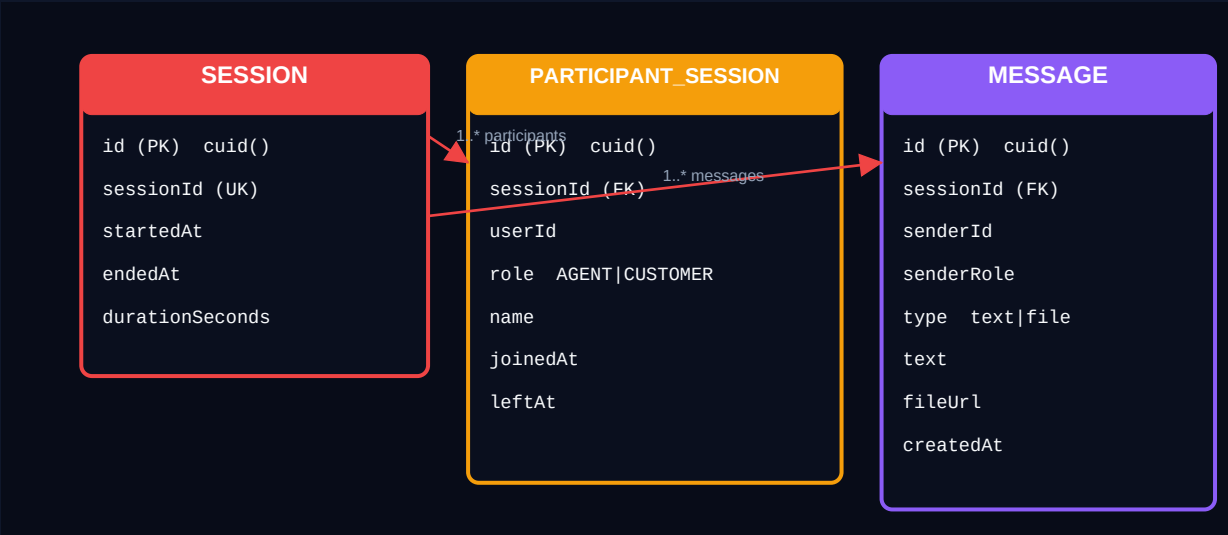
The sequence diagram below traces the critical user journeys — from session join and WebRTC negotiation, through live chat and file sharing, to AI summary generation and teardown.



Full request lifecycle from lobby authentication through WebRTC negotiation, live media, chat, file sharing, AI summarization, and graceful session teardown.

5. Database Design

SupportLens persists session metadata, participant histories, and chat transcripts across three normalized entities. All cross-entity joins use the business-key sessionId string for simplified application-layer queries.



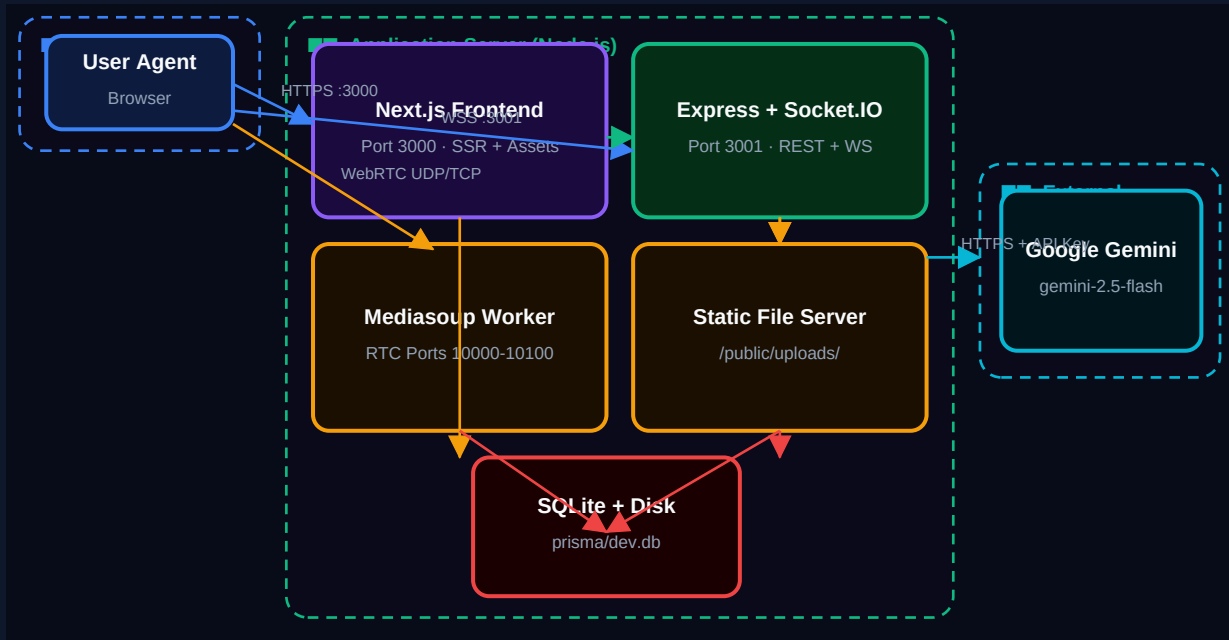
Entity-Relationship diagram. SESSION is the root aggregate; PARTICIPANT_SESSION records individual join/leave events; MESSAGE stores both text and file messages with a discriminator type field.

Index Strategy

Table	Index	Rationale
Session	sessionId (UNIQUE)	Primary lookup key for all join/socket operations
ParticipantSession	sessionId	Bulk fetches of participants per session
Message	(sessionId, createdAt)	Ordered chat history retrieval

6. Deployment Architecture

Current single-host deployment topology suitable for hackathon and local demonstration. Both the Next.js frontend and Express backend run as separate Node.js processes on the same machine. Mediasoup worker binds to the configured RTC port range for media.



Single-host deployment topology. The dashed boundary represents the target of a future containerised/cloud deployment.

7. Demo Credentials

The following credentials are pre-configured in the SupportLens lobby for demonstration purposes. Each session requires exactly one Agent and one Customer participant.

Role	Username	Password	Capabilities
■ Agent	agent	agent123	Full session control · end-call authority · AI summary generation · admin dashboard access
■ Customer	customer	customer123	Join session · video/audio/chat · file upload · graceful reconnect

■ ■ Note: Demo authentication is client-side only. Production deployments should replace this with server-side JWT/OAuth2 authentication.

8. Key Features

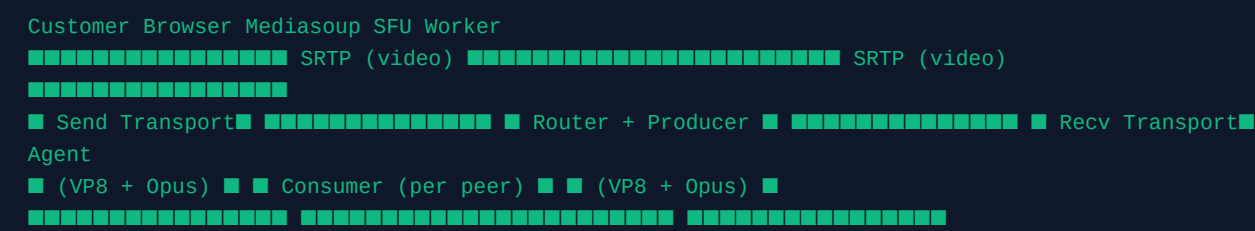
#1	■ Video Calling	HD WebRTC video (720p) routed through Mediasoup SFU with VP8 codec	Mediasoup 3.x, WebRTC
#2	■ ■ Audio Calling	Full-duplex Opus audio with mute toggle and real-time state sync	Mediasoup 3.x, WebRTC
#3	■ Real-Time Chat	Instant bidirectional messaging with delivery receipts and persistent history	Socket.IO 4.x
#4	■ File Sharing	Upload and share PNG/JPEG images and PDFs (≤10 MB) with inline preview	Multer, Express static
#5	■ Session History	Queryable log of all past sessions with participant counts and transcripts	Prisma + SQLite
#6	■ AI Session Summaries	Structured AI-generated call reports with automatic keyword-based fallback	Google Gemini API
#7	■ Admin Dashboard	Live overview: active sessions, participants, message volumes, server uptime	Next.js + REST API
#8	■ Reconnect & Recovery	Customer reconnect with media resync via getRoomProducers without page reload	Socket.IO + Mediasoup
#9	■ Observability Metrics	/api/metrics exposing live session/participant counts and uptime in JSON	Express REST
#10	■ Role Enforcement	Strict 1-Agent + 1-Customer topology enforced server-side in RoomManager	Node.js Room Manager

9. Technical Highlights

9.1 Mediasoup SFU Architecture

SupportLens routes all media through a Selective Forwarding Unit (SFU) rather than a peer-to-peer mesh. This provides three core benefits:

- Scalability:** Each participant only sends one upstream (to the SFU) regardless of participant count, keeping bandwidth $O(1)$ per sender.
- Security:** Media never traverses the application server in plaintext — SRTP encryption is negotiated end-to-end via DTLS handshake.
- Control:** The server gains full visibility and control over all media flows, enabling future recording and selective forwarding.



9.2 Socket.IO Real-Time Communication

SupportLens uses Socket.IO as its primary signaling and application-layer messaging transport. A singleton socket pattern (useSocket.ts) prevents multiple handshakes during React re-renders. Key event taxonomy:

Client → Server	<code>join</code>	Bootstrap room, load router caps + history
Client → Server	<code>createWebRtcTransport</code>	Request SFU transport parameters
Client → Server	<code>produce</code>	Publish local media track
Client → Server	<code>consume</code>	Subscribe to remote producer
Client → Server	<code>sendMessage</code>	Relay + persist chat/file message
Server → Client	<code>newProducer</code>	Notify peer of new media track
Server → Client	<code>messageReceived</code>	Fan-out persisted message to all room members
Server → Client	<code>customerDisconnected</code>	Signal agent to enter waiting state
Server → Client	<code>customerReconnected</code>	Signal agent that media resync can begin

9.3 AI Summarization Workflow

1. Fetch all Message records for sessionId (ordered by createdAt ASC)
2. Build transcript: "Alice (CUSTOMER): I can't log in\nBob (AGENT): Let me help..."
3. Construct Gemini API request — model: gemini-2.5-flash, responseSchema: {issue, resolution, status, summary}
4. Parse + validate response JSON
5. Fallback: keyword-based heuristics if API key absent or call fails

6. Return structured {issue, resolution, status, summary} to frontend

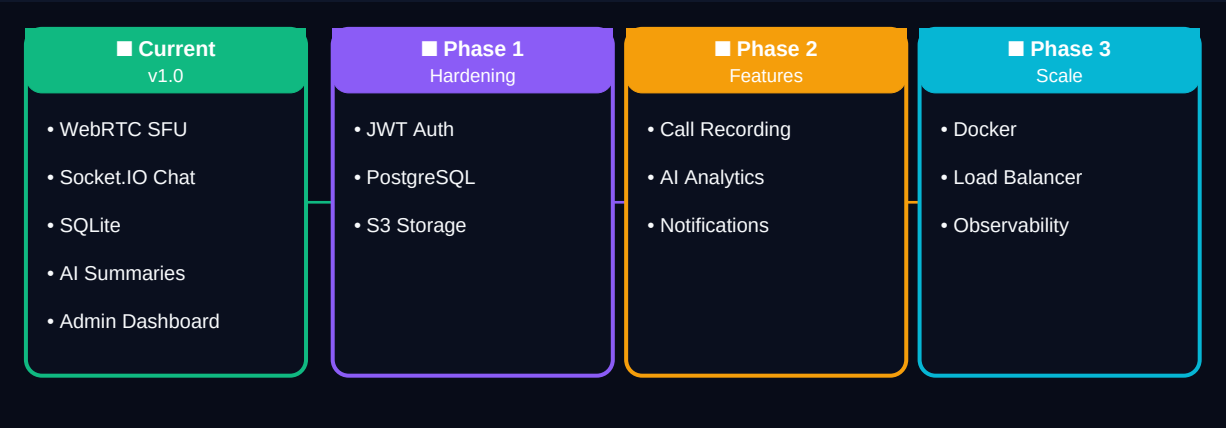
9.4 Reconnect & Recovery Mechanism

SupportLens implements asymmetric reconnect handling based on role. Customer disconnections free the room slot immediately; on rejoin, `scheduleRemoteProducerSync()` fires after a 250ms debounce, restoring remote video/audio without page reload on either side. Agent disconnections tear down the entire session via `closeRoom()`.

9.5 Session Tracking Architecture

Layer	Storage	Lifecycle
Room State	In-memory (RoomManager)	Alive only while at least one peer is connected
Session Record	SQLite Session table	Created on first join, endedAt set when room closes
Participant Record	SQLite ParticipantSession	Created on join, leftAt set on disconnect or endCall

10. Future Enhancements



Priority	Enhancement	Rationale
■ High	JWT / OAuth2 Authentication	Replace demo credentials with secure server-side token validation and role-claim verification
■ High	PostgreSQL Migration	SQLite has WAL-mode limits under concurrent write load; Prisma datasource swap requires only schema change
■ High	Cloud File Storage (S3)	Eliminate local disk dependency; enable CDN-served media with presigned upload URLs
■ Medium	Call Recording	Route SFU output to PlainRtpTransport → FFmpeg pipeline for MP4/WebM archival
■ Medium	Advanced AI Analytics	Extend Gemini prompts to produce sentiment scores, topic classification, and CSAT predictions
■ Medium	Notification System	Trigger email/SMS/webhook on session end with AI summary payload
■ Low	Docker Containerisation	Dockerfile + docker-compose.yml for reproducible deployment; Mediasoup multi-worker support
■ Low	Horizontal Scaling	Redis-backed Socket.IO adapter for multi-process signaling; Mediasoup router federation
■ Low	Full Observability Stack	Prometheus metrics endpoint, Grafana dashboards, OpenTelemetry distributed tracing

Appendix: Technology Reference

Category	Technology	Version	Purpose
Frontend Framework	Next.js	16.x	SSR + App Router + API Routes
UI Library	React	19.x	Component model + Hooks
Language	TypeScript	5.x	Type safety across full stack
State Management	Zustand	5.x	Lightweight reactive global store
Styling	Tailwind CSS	4.x	Utility-first responsive UI
Backend Framework	Express.js	5.x	REST API + middleware
WebSocket Layer	Socket.IO	4.x	Bidirectional event signaling
WebRTC SFU	Mediasoup	3.x	Server-side media routing
Mediasoup Client	mediasoup-client	3.x	Browser WebRTC SDK
ORM	Prisma	6.x	Type-safe database access
Database	SQLite	—	Embedded relational persistence
File Upload	Multer	2.x	Multipart form handling
AI Provider	Google Gemini	gemini-2.5-flash	Structured AI summarization
Process Runner	concurrently	9.x	Dev: run Next.js + server in parallel
TypeScript Runner	tsx	4.x	Execute .ts server files directly

Document prepared by the SupportLens Engineering Team · Built with ❤️ for the 2026 Hackathon